

push_retrieval_probability

January 3, 2026

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pathlib import Path
import sys
import shutil
from collections import defaultdict
from tqdm import tqdm

sys.path.insert(0, str(Path(__file__).parent.parent))
from utils import (setup_write_experiment, setup_sleep_experiment,
                    run_cpp, DATA_DIR)

PLOTS_DIR = Path(__file__).parent.parent / "plots"
PLOTS_DIR.mkdir(exist_ok=True)
```

```
[ ]: sns.set_style("darkgrid")
sns.set_context("paper", font_scale=1.5)
plt.rcParams['text.usetex'] = True
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.serif'] = ['Times']
plt.rcParams.update({
    'font.size': 20,
    'axes.labelsize': 30,
    'axes.titlesize': 30,
    'xtick.labelsize': 25,
    'ytick.labelsize': 25,
    'legend.fontsize': 20,
    'figure.titlesize': 20,
    'lines.linewidth': 2.5,
    'axes.linewidth': 1.5,
    'axes.grid': False,
    'font.weight': 'bold'
})

# Specific sizes
```

```
RHO_LABEL_SIZE = 35 # Larger size for rho row labels
```

Push vs Retrieval Probability Analysis

Objective Quantify the impact of “push” (the projection of network dynamics onto pattern directions at the neutral state) on the probability of pattern retrieval during Autonomous Retrieval (AR).

Experimental Design - **Network size**: $N = 200$ neurons - **Sparsity**: $s = 0.5$ ($P(0)$ convention, fraction of inactive units) - **Correlation**: $\rho = 0.5$ (dynamics are qualitatively similar across correlation and β values explored in earlier simulations) - **Memory loads**: $p \in \{2, 3, 4, 5, 7, 10\}$ patterns - **Repetitions**: N_REPS independent networks per configuration

Push Metric with Inhibition The push $P_\mu(t)$ for pattern μ at iteration t is defined as:

$$P_\mu(t) = \langle f_t, \hat{d}_\mu \rangle$$

where: - $f_t = -v_N + \sigma(W \cdot v_N) - A_{t-1} \odot v_N$ is the network derivative at neutral state, incorporating the diagonal inhibition matrix from the previous iteration - $\hat{d}_\mu = (x^\mu - v_N) / \|x^\mu - v_N\|$ is the unit direction toward pattern μ - A_t evolves during AR as patterns are retrieved and inhibited

Key point: Push values and pattern rankings are **recomputed at each iteration** using the inhibition state BEFORE the current retrieval (i.e., A_{t-1} for iteration t , with $A_{-1} = 0$ for the first iteration). This captures how inhibitory plasticity dynamically reshapes the push landscape during AR.

Main Finding The push from the neutral state is a good predictor of which pattern will be retrieved, as long as the memory load stays small. When approaching the critical load, push becomes a worse predictor. This corresponds to loads where spurious attractors start to appear more frequently, as shown in previous simulations.

Analysis Pipeline 1. Train network on p patterns with correlation ρ (parallel across seeds) 2. Run AR until first spurious attractor or all patterns found 3. **At each iteration t** : load inhibition matrix A_{t-1} (state before retrieval), recompute push values, and re-rank patterns (rank 0 = highest current push) 4. Track which rank was retrieved at each AR iteration 5. Aggregate statistics across repetitions

```
[ ]: RHOS = [0.5]
LOADS = [2,3,4,5,7,10]
N = 200
SPARSITY = 0.5
N_REPS = 10
# Sigmoid function
sigma = lambda x: 1.0 / (1.0 + np.exp(-x))
```

```
[ ]: # For Figure 1: retrieval_counts[rho_idx][load_idx][rank] = count
retrieval_counts = [[defaultdict(int) for _ in LOADS] for _ in RHOS]
total_retrievals = [[0 for _ in LOADS] for _ in RHOS]

# For Figure 2: iteration_data[rho_idx][load_idx][iteration] = list of bools
# ↪ (was rank 0 retrieved?)
```

```
iteration_data = [[defaultdict(list) for _ in LOADS] for _ in RHOS]
```

```
[ ]: total_configs = len(RHOS) * len(LOADS)
DEBUG_FIRST = False # Set True to enable debug output for first config

with tqdm(total=total_configs, desc="Configurations", unit="cfg",
          bar_format="{l_bar}{bar}| {n_fmt}/{total_fmt} [{elapsed}<{remaining}]_
→{postfix}") as pbar:

    for rho_idx, rho in enumerate(RHOS):
        for load_idx, load in enumerate(LOADS):
            pbar.set_postfix_str(f"rho={rho}, p={load}")
            is_first_config = (rho_idx == 0 and load_idx == 0)

            sim_name = f"push_prob_rho{rho}_load{load}"

            # Clean up old simulation folders to avoid stale data contamination
            train_dir = DATA_DIR / "trained_networks" / sim_name
            sleep_dir_base = DATA_DIR / "sleep_results" / sim_name
            if train_dir.exists():
                shutil.rmtree(train_dir)
            if sleep_dir_base.exists():
                shutil.rmtree(sleep_dir_base)

            # Train networks in parallel using native pattern generation
            write_config = setup_write_experiment(
                name=sim_name,
                patterns=None, # C++ generates patterns internally
                params={
                    "network_size": N,
                    "num_patterns": load,
                    "sparsity": SPARSITY,
                    "rho": rho,
                    "leak": 1.0,
                    "drive_target": 6.0,
                    "learning_rate": 0.0001,
                    "momentum": 0.9,
                    "max_iterations": 5000,
                    "convergence_threshold": 0.01
                },
                varying_params={"seed": list(range(N_REPS))},
                native_pattern_generation=True
            )
            run_cpp("write", write_config, verbose=False)

            # Run AR in parallel for all repetitions (noise disabled for
            →deterministic push analysis)
```

```

sleep_config = setup_sleep_experiment(
    name=sim_name,
    trained_networks_dir=train_dir,
    params={
        "beta": 0.0001,
        "delta": 0.01,
        "max_queries": load * 2,
        "stop_on_spurious": 1,
        "stop_on_all_found": 1,
        "save_inhibition_matrices": 1,
        "noise_dynamics": 0 # Disable noise for deterministic
    }
)
run_cpp("sleep", sleep_config, verbose=False)

# Collect results from all repetitions
for rep in range(N_REPS):
    sim_dir = train_dir / f"sim_nb_{rep}"
    sleep_dir = sleep_dir_base / f"sim_nb_{rep}"

    if not sim_dir.exists() or not sleep_dir.exists():
        continue

    W = np.loadtxt(sim_dir / "weights.data")
    X_raw = np.loadtxt(sim_dir / "patterns.data")

    # Debug: print raw shape
    if DEBUG_FIRST and is_first_config and rep == 0:
        print(f"\n=== DEBUG: First config, first rep ===")
        print(f"Raw X shape: {X_raw.shape}")
        print(f"Expected: ({load}, {N})")

    # Fix pattern matrix orientation: ensure (n_patterns, N)
    X = X_raw
    if X.shape != (load, N):
        X = X.T
        if DEBUG_FIRST and is_first_config and rep == 0:
            print(f"Transposed X to shape: {X.shape}")
    assert X.shape == (load, N), f"Unexpected pattern shape: {X.
    shape}"

    results = pd.read_csv(sleep_dir / "results.data")
    v_N = np.ones(N) * 0.5
    recovered_indices = results["recovered_pattern_idx"].values

    # Precompute pattern directions (normalized)

```

```

        d_hat = np.array([(X[mu] - v_N) / np.linalg.norm(X[mu] - v_N)]
→for mu in range(load)])

        # Process each iteration with updated push values
        for t, pat_idx in enumerate(recovered_indices):
            if pat_idx < 0 or pat_idx >= load: # Spurious or invalid
                continue

            # Load inhibition matrix from PREVIOUS iteration (state
→BEFORE retrieval at t)
            # inhib_matrix_iter_{t}.data contains state AFTER iteration
→t

            # So for iteration t, we need inhib_matrix_iter_{t-1}.data
            # For t=0, there's no previous inhibition (use zeros)
            if t == 0:
                A_diag = np.zeros(N)
            else:
                inhib_file = sleep_dir / f"inhib_matrix_iter_{t-1}.data"
                if not inhib_file.exists():
                    continue
                A = np.loadtxt(inhib_file)
                A_diag = np.diag(A)

            # Compute push with current inhibition state
            f_t = -v_N + sigma(W @ v_N) - A_diag * v_N
            P_t = np.array([np.dot(f_t, d_hat[mu]) for mu in
→range(load)])

            # Rank patterns by current push (highest = rank 0)
            push_ranks_t = np.argsort(-P_t) # Descending order
            pattern_to_rank_t = {pat: rank for rank, pat in
→enumerate(push_ranks_t)}

            rank = pattern_to_rank_t[pat_idx]

            # Debug: show push values and ranking for first iteration
            if DEBUG_FIRST and is_first_config and rep == 0 and t == 0:
                print(f"\n--- Iteration {t} ---")
                print(f"W shape: {W.shape}")
                print(f"A (inhibition) sum: {np.sum(A_diag)} (should be
→0 at t=0)")

                print(f"Push values P_t: {P_t}")
                print(f"argsort(-P_t) = {push_ranks_t}")
                print(f"Pattern with highest push: {push_ranks_t[0]}
→(P={P_t[push_ranks_t[0]]:.4f})")

```

```

        print(f"Pattern with lowest push: {push_ranks_t[-1]}")
→(P={P_t[push_ranks_t[-1]]:.4f})")
        print(f"Retrieved pattern: {pat_idx}")
→(push={P_t[pat_idx]:.4f}, rank={rank})")
        print(f"Pattern 0 sparsity (P(0)): {1 - np.mean(X[0]):.
→3f}")
        print(f"Pattern 1 sparsity (P(0)): {1 - np.mean(X[1]):.
→3f}")

        # Figure 1: count retrievals by rank (using current ranking)
        retrieval_counts[rho_idx][load_idx][rank] += 1
        total_retrievals[rho_idx][load_idx] += 1

        # Figure 2: was the current highest-push pattern (rank 0)
→retrieved?

        iteration_data[rho_idx][load_idx][t].append(rank == 0)

    pbar.update(1)

# Debug: Verify per-iteration data for load=10
if 10 in LOADS:
    load_idx_10 = LOADS.index(10)
    print(f"\n=== DEBUG: Per-iteration data for load=10 ===")
    for t in sorted(iteration_data[0][load_idx_10].keys()):
        data = iteration_data[0][load_idx_10][t]
        if data:
            print(f"Iter {t}: samples={data}, n={len(data)}, mean={np.
→mean(data):.2f}")

```

Configurations: 100%| | 6/6 [00:24<00:00] , rho=0.5, p=10

```

=== DEBUG: Per-iteration data for load=10 ===
Iter 0: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 1: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 2: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 3: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 4: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 5: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 6: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40

```

```

Iter 7: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 8: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 9: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 10: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 11: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 12: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 13: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 14: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 15: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 16: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 17: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 18: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40
Iter 19: samples=[False, True, False, True, False, False, True, True, False,
False], n=10, mean=0.40

```

```

[ ]: # Compute probabilities from counts
retrieval_probs = [{ } for _ in LOADS] for _ in RHOS

for rho_idx in range(len(RHOS)):
    for load_idx, load in enumerate(LOADS):
        total = total_retrievals[rho_idx][load_idx]
        if total > 0:
            for rank in range(load):
                count = retrieval_counts[rho_idx][load_idx][rank]
                retrieval_probs[rho_idx][load_idx][rank] = count / total

```

Figure 1: Retrieval Probability by Push Rank

Bar/dot charts showing the probability that the retrieved pattern had a given push rank **at the moment of retrieval** (accounting for current inhibition state).

Important: Ranks are recomputed at each iteration using the current inhibition matrix A_t . This means rank 0 always refers to the pattern with the highest push *at that moment*, not a fixed initial ranking.

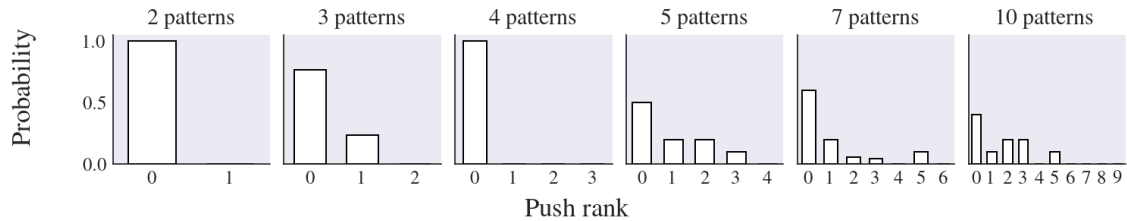
Interpretation: At low loads, rank 0 (highest push) strongly dominates, confirming that push is a

good predictor. As load increases toward the critical capacity, the distribution flattens, indicating push becomes less predictive. This transition coincides with the emergence of spurious attractors.

```
[ ]: fig1, axes1 = plt.subplots(len(RHOS), len(LOADS), figsize=(23, 3.5 *  
    ↳len(RHOS)), sharex=False, sharey=True, squeeze=False)  
  
for rho_idx, rho in enumerate(RHOS):  
    for load_idx, load in enumerate(LOADS):  
        ax = axes1[rho_idx, load_idx]  
  
        ranks = list(range(load))  
        probs = [retrieval_probs[rho_idx][load_idx].get(r, 0) for r in ranks]  
  
        bars = ax.bar(ranks, probs, facecolor='white', edgecolor='black',  
    ↳linewidth=2, width=0.6)  
  
        # Title for top row only  
        if rho_idx == 0:  
            ax.set_title(f'{load} patterns', pad=15)  
  
        # Rho annotation for left column (shifted left to avoid overlap with  
    ↳shared label)  
        # if load_idx == 0:  
        #     ax.annotate(f'$\\rho={rho}$', xy=(5.9, 0.5), xycoords='axes  
    ↳fraction',  
        #                 fontsize=RHO_LABEL_SIZE, ha='left', va='center',  
    ↳rotation=0)  
  
        ax.set_xticks(ranks)  
        ax.set_xlim(-0.5, load - 0.5)  
  
        for spine in ['bottom', 'left']:  
            ax.spines[spine].set_visible(True)  
            ax.spines[spine].set_color('black')  
            ax.spines[spine].set_linewidth(1.5)  
        for spine in ['top', 'right']:  
            ax.spines[spine].set_visible(False)  
  
plt.tight_layout()  
fig1.subplots_adjust(left=0.18, bottom=0.1) # Make room for rho annotations  
    ↳and shared labels  
  
# Add shared axis labels  
fig1.supxlabel('Push rank', fontsize=35, fontweight='bold', x = 0.55, y = -0.2 )  
fig1.supylabel('Probability', fontsize=35, fontweight='bold', x=0.1)
```

```
plt.savefig(PLOTS_DIR / 'push_retrieval_probability.png', dpi=300,
            ↳bbox_inches='tight')
print(f"Saved to {PLOTS_DIR / 'push_retrieval_probability.png'}")
plt.show()
```

Saved to /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/scripts/plots/push_retrieval_probability.png



```
[ ]: fig1b, axes1b = plt.subplots(len(RHOS), len(LOADS), figsize=(23, 3.5 *
            ↳len(RHOS)), sharex=False, sharey=True, squeeze=False)

for rho_idx, rho in enumerate(RHOS):
    for load_idx, load in enumerate(LOADS):
        ax = axes1b[rho_idx, load_idx]

        ranks = list(range(load))
        probs = [retrieval_probs[rho_idx][load_idx].get(r, 0) for r in ranks]

        # Dot plot with black edge, white fill
        ax.scatter(ranks, probs, s=200, facecolor='white', edgecolor='black',
            ↳linewidth=2, zorder=3)

        # Add horizontal line at y=0 for reference
        ax.axhline(y=0, color='black', linewidth=1, linestyle='-', alpha=0.3)

        # Title for top row only
        if rho_idx == 0:
            ax.set_title(f'{load} patterns', pad=15)

        ax.set_xticks(ranks)
        ax.set_xlim(-0.5, load - 0.5)
        ax.set_ylim(-0.2, 1.2) # Extend y-axis below 0

        for spine in ['bottom', 'left']:
            ax.spines[spine].set_visible(True)
            ax.spines[spine].set_color('black')
            ax.spines[spine].set_linewidth(1.5)
        for spine in ['top', 'right']:
```

```

ax.spines[spine].set_visible(False)

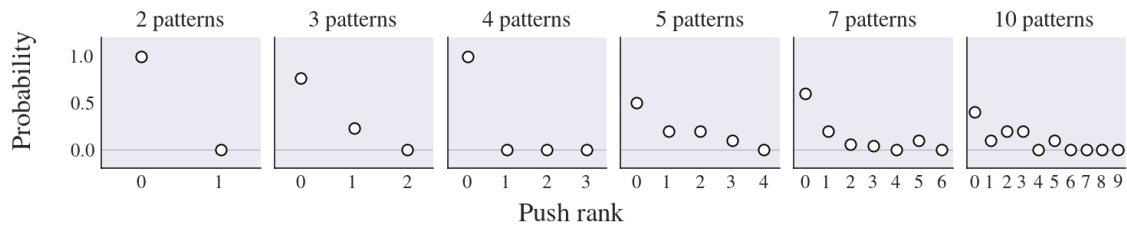
plt.tight_layout()
fig1b.subplots_adjust(left=0.18, bottom=0.1)

# Add shared axis labels
fig1b.supxlabel('Push rank', fontsize=35, fontweight='bold', x=0.55, y=-0.2)
fig1b.supylabel('Probability', fontsize=35, fontweight='bold', x=0.11)

plt.savefig(PLOTS_DIR / 'push_retrieval_probability_dots.png', dpi=300,
            bbox_inches='tight')
print(f"Saved to {PLOTS_DIR / 'push_retrieval_probability_dots.png'}")
plt.show()

```

Saved to /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/scripts/plots/push_retrieval_probability_dots.png



```

[ ]: load_idx_10 = LOADS.index(10)

# Debug: print raw data
print(f"\n=== Per-iteration breakdown for load=10 ===")
for t in sorted(iteration_data[0][load_idx_10].keys()):
    data = iteration_data[0][load_idx_10][t]
    print(f"Iter {t}: {data} -> mean={np.mean(data):.2f}")

fig3, ax3 = plt.subplots(figsize=(10, 6))

iterations = sorted(iteration_data[0][load_idx_10].keys())
probs_by_iter = []
for t in iterations:
    data = iteration_data[0][load_idx_10][t]
    if len(data) > 0:
        probs_by_iter.append((t, np.mean(data), len(data)))

if probs_by_iter:
    iters, probs, counts = zip(*probs_by_iter)

    ax3.plot(iters, probs, 'o-', color='black', linewidth=2, markersize=8,

```

```

        markerfacecolor='white', markeredgewidth=2,
        ↪markeredgewidth=2)

    # Annotate sample counts
    for i, (it, p, n) in enumerate(probs_by_iter):
        ax3.annotate(f'n={n}', (it, p), textcoords="offset points",
                    xytext=(0, 10), ha='center', fontsize=10)

ax3.set_xlabel('AR iteration')
ax3.set_ylabel('P(highest push retrieved)')
ax3.set_title('Load = 10 patterns')
ax3.set_ylim(0, 1.05)

for spine in ['top', 'right']:
    ax3.spines[spine].set_visible(False)

plt.tight_layout()
plt.savefig(PLOTS_DIR / 'push_retrieval_probability_evolution.png', dpi=300,
    ↪bbox_inches='tight')
print(f"Saved to {PLOTS_DIR / 'push_retrieval_probability_evolution.png'}")
plt.show()

```

=== Per-iteration breakdown for load=10 ===

```

Iter 0: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 1: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 2: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 3: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 4: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 5: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 6: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 7: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 8: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 9: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 10: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 11: [False, True, False, True, False, False, True, True, False, False] ->

```

```

mean=0.40
Iter 12: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 13: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 14: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 15: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 16: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 17: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 18: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Iter 19: [False, True, False, True, False, False, True, True, False, False] ->
mean=0.40
Saved to /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulat
ions/main_write_sleep/scripts/plots/push_retrieval_probability_evolution.png

```

